

DATA SCIENCE EXAM CHEATSHEET

Unsupervised Learning: K-Means Clustering & PCA

1. Pre-processing & Scaling

Why Standardize? K-Means calculates physical distance (like Euclidean distance) between data points. If features have drastically different scales (e.g., Price vs. Age), the larger numbers will mathematically dominate. Standardizing puts all features on an equal playing field.

- **Tree-based algorithms** (Random Forest) do NOT need scaling.
- **Distance-based algorithms** (K-Means, PCA, KNN) **REQUIRE** scaling.

```
from sklearn.preprocessing import
StandardScaler
features =
df.select_dtypes(include=['float64',
'int64'])
scaler = StandardScaler()
scaled_features =
scaler.fit_transform(features)
```

2. K-Means Clustering

Goal: Group unlabeled data into k distinct natural clusters.

Finding the Optimal k

We use a loop to test different values of k . We score each run using the **Silhouette Score** (ranges from -1 to 1). A score closer to 1 means the clusters are dense and well-separated.

```
best_k = 2
best_score = -1
for k in range(2, 10):
    kmeans_test = KMeans(n_clusters=k,
random_state=42)
    labels =
kmeans_test.fit_predict(scaled_features)
    score =
silhouette_score(scaled_features,
labels)

    if score > best_score:
        best_score = score
        best_k = k
```

Once optimal k is found, run the final model and attach labels:

```
final_model = KMeans(n_clusters=best_k)
df['Cluster'] =
final_model.fit_predict(scaled_features)
```

3. Principal Component Analysis (PCA)

Goal: Dimensionality reduction (The "Data Squisher"). It takes high-dimensional data and compresses it into fewer dimensions (e.g., 2D) so humans can visualize the clusters on a flat graph.

- **Principal Components:** The new columns created by mixing the original features together mathematically.
- **Explained Variance:** The "report card." It tells you what percentage of the original dataset's mathematical spread was successfully retained in the new dimensions.

```
from sklearn.decomposition import PCA
# Reduce to 2 dimensions for plotting
pca = PCA(n_components=2)
pca_transformed =
pca.fit_transform(scaled_features)

# Print the report card
print(pca.explained_variance_ratio_)
```

Check the Shapes: The original dataset might be **(100, 5)**. The transformed dataset will be **(100, 2)**.

4. Plotting the Results

Use seaborn to plot the squished 2D data (**PC1** vs **PC2**) and color the dots using the K-Means cluster labels.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create temporary DataFrame for plotting
pca_df = pd.DataFrame(pca_transformed,
                      columns=['PC1',
                              'PC2'])
pca_df['Cluster'] = df['Cluster'] # Add K-Means labels

plt.figure(figsize=(8, 5))
sns.scatterplot(x='PC1', y='PC2',
               hue='Cluster',
               data=pca_df,
               palette='viridis')
plt.show()
```

5. Common Python Pitfalls

TypeError: 'module' object is not callable

Caused by importing the wrong level of Matplotlib.

Wrong: import matplotlib as plt

Correct: import matplotlib.pyplot as plt

NameError: name 'kmeans_test' is not defined

Python is case-sensitive! Ensure variables like `Kmeans_test` and `kmeans_test` are typed exactly the same way when defined and when called.